

# A New Input Grouping and Sharing Method to Design Low Complexity FFT Implementation

Shouman Wang and Jiajia Chen<sup>✉</sup>

**Abstract**—FFT is a commonly applied algorithm in digital signal processing and communications. In this brief, a new low power and low complexity FFT architecture design is proposed. An input grouping method is used to reduce the multiplications of the inputs and FFT twiddle factor coefficients. In addition, a new input partial sum sharing scheme is proposed to reuse the hardware resources to further reduce the adder cost. Logic synthesis results in ASIC show that the proposed 16-point FFT architecture can save area and power cost by at least 19.1% and 19.0% respectively compared with the recently published designs. Similarly, the proposed 32-point FFT architecture can reduce both power and delay by at least 6.91% and 5.35%.

**Index Terms**—FFT, twiddle factors, partial sum sharing.

## I. INTRODUCTION

**F**AST Fourier transform (FFT) is an essential algorithm in many applications such as the fifth generation (5G) wireless communication [1], digital signal processing systems [2], scientific computing operations [3], [4], etc. Cooley-Tukey FFT algorithm [5] firstly exploits the symmetry and periodicity properties of DFT to improve the computation.

FFT was commonly implemented in software. However, with the development of system-on-a-chip (SoC) and other technologies, FFT implementation on hardware based becomes possible to further reduce area and power under the circumstances of maintaining output throughput.

Existing FFT architectures can be mainly categorized into two types, memory-based architectures [6]–[8] and pipelined architectures [9]–[11]. The former type uses only one butterfly processing element (PE) and memory blocks for storing intermediate data. It saves computing resources, but data throughput rate is lower. For applications which require high throughputs, such as IEEE 802.11ad [12], pipelined architectures having separate PEs and memory for each stage can achieve high throughputs. These pipelined architectures can be further classified into single-path delay feedback (SDF) [13], [14], multi-path delay feedback (MDF) [15] and multi-path delay commutator (MDC) [16]. Due to the need for more butterfly PEs and FFT rotators to perform twiddle factor multiplications, the hardware cost of pipelined architectures is significantly increased. In order to solve the problem, canonical signed digit (CSD) arithmetic and common subexpression

sharing methods [17] are proposed to reduce the number of adders. In [17], an efficient parallel FFT architecture is designed by truncating its insignificant bits. Another approximated parallel FFT architecture was recently proposed in [18] where a new twiddle factor merging technique is proposed to implement the merged expressions. Although a low area-power design is achieved in [18], more efficient hardware solution within the margin of error can be further developed.

FFT can also be implemented using fixed-point or floating-point FFT. Some fixed-point implementation [19] adjusts the word-length to achieve tradeoff between accuracy and complexity. In [9], the output word-length of the multiplier is rounded to the same length of the input to reduce complexity. Some floating-point architecture [20] is based on the adaptive CORDIC. It can achieve higher accuracy at the expense of complexity through higher iteration number. Generally, floating-point FFT is more advantageous for mathematical calculations and avoiding overflow/underflow, benefited from high dynamic range. On the other hand, despite the accuracy due to the limited word-length, the fixed-point FFT has the advantages of lower hardware complexity. Therefore, in this brief, we adopt the fixed-point implementation.

In this brief, we propose a new area-power efficient FFT design method, with the following two new contributions.

1. We propose a new input grouping method to reduce the multiplier cost. Unlike the existing Decimation in Time, the particular inputs multiplying with the same twiddle factors are grouped such that only trivial factors appear inside the group and they requires no multiplier.

2. After grouping, we propose a new input partial sum sharing algorithm to reduce the adder cost. The input partial sums with greater probability of sharing are found, which can be reused in the same group to save adders.

## II. RELATED WORKS

### A. DFT and FFT Preliminaries

The  $N$ -point DFT  $X(k)$  [21] is defined by

$$X(k) = \sum_{n=0}^{N-1} x(n)W_N^{nk}, \quad k = 0, 1, \dots, N-1, \quad (1)$$

where  $X(k)$  is the frequency domain representation of  $x(n)$ .  $x(n)$  is the input sequence in time domain.  $W_N^{nk} = \exp(-j2\pi nk/N)$  is known as twiddle factor [21].

Computing DFT directly by using (1) requires  $N^2$  multiplications and  $N(N-1)$  additions, which are heavy especially when  $N$  increases. Therefore, FFT was proposed to compute DFT efficiently. Cooley-Tukey radix-2 decimation-in-time (DIT) FFT is the simplest and most commonly algorithm as

$$X(k) = X_1(k) + W_N^k X_2(k), \quad (2)$$

Manuscript received 19 June 2022; accepted 30 June 2022. Date of publication 4 July 2022; date of current version 7 February 2024. This work was supported by the Nanjing University of Aeronautics and Astronautics, Nanjing, China, under Grant 56YAH18043 and Grant PAC19009. This brief was recommended by Associate Editor G. Jovanovic Dolecek. (Corresponding author: Jiajia Chen.)

The authors are with the College of Electronic and Information Engineering, Nanjing University of Aeronautics and Astronautics, Nanjing 210016, China (e-mail: tim\_king20@hotmail.com).

Color versions of one or more figures in this article are available at <https://doi.org/10.1109/TCSII.2022.3188255>.

Digital Object Identifier 10.1109/TCSII.2022.3188255

1549-7747 © 2022 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.

See <https://www.ieee.org/publications/rights/index.html> for more information.

Authorized licensed use limited to: Rainy K. Downloaded on August 14, 2024 at 05:50:40 UTC from IEEE Xplore. Restrictions apply.

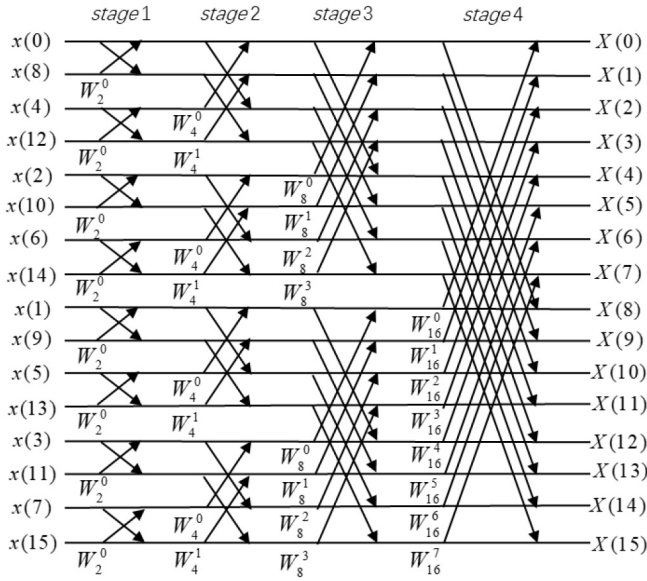
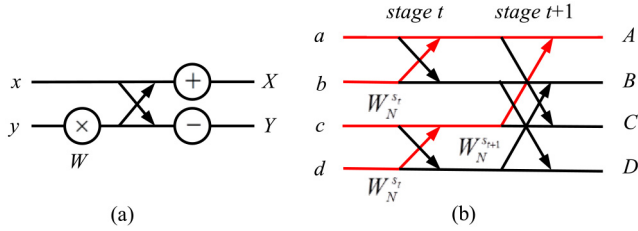


Fig. 1. 16-point DIT-FFT algorithm.

Fig. 2. (a) The butterfly block, (b) Multiplication of twiddle factors across stages in an  $N$ -point radix-2 FFT architecture.

where  $X_1(k) = \sum_{n=0}^{N/2-1} x(2n)W_{N/2}^{nk}$  and  $X_2(k) = \sum_{n=0}^{N/2-1} x(2n+1)W_{N/2}^{nk}$  are two  $N/2$ -point DFTs. By recursively applying (2), an  $N$ -point DFT can be decomposed into  $\log_2 N$  stages and each stage contains a total of  $N/2$  2-point DFTs. A computation scheme of a 16-point DIT-FFT consisting of 4 stages is depicted in Fig. 1. The butterfly block depicted in Fig. 2(a) is utilized to implement these 2-point DFTs based on the symmetric property of the twiddle factors which is

$$W_N^{nk+N/2} = -W_N^{nk}. \quad (3)$$

In each butterfly block, one complex multiplication, one addition and one subtraction are required. Therefore, the computational complexity of an  $N$ -point FFT decreases from  $O(N^2)$  to  $O(N \log_2 N)$ .

### B. Twiddle Factor Merging

Using twiddle factor merging method [18], we merge  $W_N^{s_t}$  and  $W_N^{s_{t+1}}$  into  $W_N^{s_t+s_{t+1}}$  based on the conventional radix-2 DIT-FFT architecture, as shown in Fig. 2(b).  $a, b, c, d$  are four intermediate values in FFT.  $A$  is computed as:

$$\begin{aligned} A &= a + W_N^{s_t} b + W_N^{s_{t+1}} (c + W_N^{s_t} d) \\ &= a + W_N^{s_t} b + W_N^{s_t+s_{t+1}} c + W_N^{s_t} W_N^{s_{t+1}} d. \end{aligned} \quad (4)$$

Therefore, when  $s_{t+1} = N/4 - (s_t + s_{t+1})$ , (4) is decomposed into multiplications with twiddle factor coefficients as

$$\begin{aligned} A &= (a + \cos \frac{2\pi s_t}{N} b + \cos \frac{2\pi s_{t+1}}{N} c + \sin \frac{2\pi s_{t+1}}{N} d) \\ &\quad - j(\sin \frac{2\pi s_t}{N} b + \sin \frac{2\pi s_{t+1}}{N} c + \cos \frac{2\pi s_{t+1}}{N} d). \end{aligned} \quad (5)$$

## III. THE PROPOSED METHOD

### A. Input Grouping

Multiplication is the most resource-expensive arithmetic operation, so reducing the number of multipliers can greatly reduce the cost. Benefitted from the twiddle factor merging in (5), FFT output is eventually expressed as the sum of the products of twiddle factors and the inputs. Therefore, we propose an input regrouping method. When different inputs are multiplied with the twiddle factors which have the same absolute value, we can first group them to perform addition or subtraction. The sum or difference is subsequently multiplied with the common twiddle factor. Additional buffers are employed to store the inputs which arrive earlier, but the saving from the reduction of multipliers is more significant.

For clear illustration, we take 16-point FFT as an example. By recursively using (2), the computation of 16-point FFT is expressed as:  $X(k) = (x(0) + W_2^k x(8)) + W_4^k [x(4) + W_2^k x(12)] + W_8^k [(x(2) + W_2^k x(10)) + W_4^k (x(6) + W_2^k x(14))] + W_{16}^k [(x(1) + W_2^k x(9)) + W_4^k ((x(5) + W_2^k x(13))) + W_8^k ((x(3) + W_2^k x(11)) + W_4^k ((x(7) + W_2^k x(15))))]$ , where  $X(k)$  is the  $k$ th output in frequency domain. By using Euler's rule and the twiddle factor merging in (5),  $X(k)$  can then be expressed as:

$$\begin{aligned} X(k) &= (x(0) + W_2^k x(8)) + \cos \frac{k\pi}{2} [x(4) + W_2^k x(12)] + \cos \frac{k\pi}{4} [x(2) + W_2^k x(10)] \\ &\quad + \cos \frac{3k\pi}{4} [x(6) + W_2^k x(14)] + \cos \frac{k\pi}{8} [x(1) + W_2^k x(9)] + \cos \frac{7k\pi}{8} \\ &\quad [x(7) + W_2^k x(15)] + \cos \frac{5k\pi}{8} [x(5) + W_2^k x(13)] + \cos \frac{3k\pi}{8} [x(3) + W_2^k x(11)] \\ &\quad - j[\sin \frac{k\pi}{2} [x(4) + W_2^k x(12)] + \sin \frac{k\pi}{4} [x(2) + W_2^k x(10)] + \sin \frac{3k\pi}{4} \\ &\quad [x(6) + W_2^k x(14)] + \sin \frac{k\pi}{8} [x(1) + W_2^k x(9)] + \sin \frac{7k\pi}{8} [x(7) + W_2^k x(15)] \\ &\quad + \sin \frac{5k\pi}{8} [x(5) + W_2^k x(13)] + \sin \frac{3k\pi}{8} [x(3) + W_2^k x(11)]] \end{aligned} \quad (6)$$

where  $W_2^k$  is equal to 1 or  $-1$ . Multiplying the input with  $-1$  or 1 can be implemented by direct hardwiring. Based on the properties of symmetry in trigonometric functions as

$$\begin{aligned} \cos \frac{k\pi}{4} &= W_2^k \cos \frac{3k\pi}{4}, \cos \frac{k\pi}{8} = W_2^k \cos \frac{7k\pi}{8}, \cos \frac{3k\pi}{8} = W_2^k \cos \frac{5k\pi}{8}, \\ \sin \frac{k\pi}{4} &= -W_2^k \sin \frac{3k\pi}{4}, \sin \frac{k\pi}{8} = -W_2^k \sin \frac{7k\pi}{8}, \sin \frac{3k\pi}{8} = -W_2^k \sin \frac{5k\pi}{8}, \end{aligned} \quad (7)$$

we group those inputs which multiply with the same twiddle factors. With (6) and (7), we divide  $x(n)$  ( $n = 0, 1, \dots, 15$ ) into four groups as follows:  $\{x(0), x(8), x(4), x(12)\}$ ;  $\{x(2), x(6), x(10), x(14)\}$ ;  $\{x(1), x(7), x(9), x(15)\}$ ;  $\{x(3), x(5), x(11), x(13)\}$ . Therefore, (7) can be rewritten as:

$$\begin{aligned} X(k) &= (x(0) + W_2^k x(8)) + \cos \frac{k\pi}{2} [x(4) + W_2^k x(12)] + \cos \frac{k\pi}{4} [x(2) + W_2^k x(10)] \\ &\quad + W_2^k (x(6) + W_2^k x(14))] + \cos \frac{k\pi}{8} [x(1) + W_2^k x(9) + W_2^k (x(7) + W_2^k x(15))] \\ &\quad + \cos \frac{3k\pi}{8} [x(3) + W_2^k x(11) + W_2^k (x(5) + W_2^k x(13))] - j[\sin \frac{k\pi}{2} [x(4) + W_2^k x(12)] \\ &\quad + \sin \frac{k\pi}{4} [x(2) + W_2^k x(10) - W_2^k (x(6) + W_2^k x(14))] + \sin \frac{k\pi}{8} [x(1) + W_2^k x(9) \\ &\quad - W_2^k (x(7) + W_2^k x(15))] + \sin \frac{3k\pi}{8} [x(3) + W_2^k x(11) - W_2^k (x(5) + W_2^k x(13))] \end{aligned} \quad (8)$$

As shown in (8), the inputs in the group still need to be multiplied by the twiddle factor  $W_2^k$  which is trivial, so no multiplier is needed. By this, at most six multipliers are needed to compute one output  $X(k)$ , whereas twelve multipliers

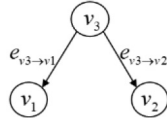


Fig. 3. Digraph representation.

are needed in original 16-point DIT-FFT architecture. For example, the computation of  $X(1)$  is finally expressed as:

$$\begin{aligned} X(1) = & (x(0)-x(8)) + \cos \frac{\pi}{4} [(x(2)-x(10)) - (x(6)-x(14))] + \cos \frac{\pi}{8} [(x(1) \\ & - x(9)) - (x(7)-x(15))] + \cos \frac{3\pi}{8} [(x(3)-x(11)) - (x(5)-x(13))] - j \{ (x(4) \\ & - x(12)) + \sin \frac{\pi}{4} [(x(2)-x(10)) + (x(6)-x(14))] + \sin \frac{\pi}{8} [(x(1)-x(9)) \\ & + (x(7)-x(15))] + \sin \frac{3\pi}{8} [(x(3)-x(11)) + (x(5)-x(13))] \}. \end{aligned}$$

Other outputs with  $k = 0, 2, 3, \dots, 15$  can be computed by (8).

### B. Input Partial Sum Sharing

Because the grouped inputs are added or subtracted before multiplications and the partial sums or differences can be shared in the same group, we propose an input partial sum (IPS) sharing method to reduce the adder cost. Weight- $N$  IPS denotes the sum or difference of  $N$  inputs. If one IPS appears more than once in the same group, it is considered as common.

There are more than two inputs in the group, so fixing one IPS means the inputs belong to this IPS are added together first. Therefore, they cannot be added with other inputs which are not inside this IPS. Eventually, the problem is how to generate efficient common IPSs which can maximize the sharing. In order to describe the IPS sharing vividly, we model it as a binary tree where the roots correspond to high-weight IPSs to be split and the leaves are the weight-two IPSs. This tree structure can be considered as a directed-acyclic graph (DAG)  $G = \{V, E\}$  where  $V$  is a set of vertices and  $E$  represents a set of directed edges. Let  $u \rightarrow v$  be an edge  $e \in E$  directed from a parent vertex  $u$  to a children vertex  $v$ . Since two inputs,  $v_1$  and  $v_2$ , are added or subtracted to produce weight-two IPS, the sign of the IPS will be encoded as follows:  $e_{u \rightarrow v} = \{sgn\}$ , where  $sgn$  is a unary sign operator used to add and subtract  $u$  to produce  $v$ . Fig. 3 shows the operation that the parent vertex  $v_3$  is split into two children vertices,  $v_1$  and  $v_2$ .

The parent vertex,  $v_3$ , is related to the children vertices as follows:

$$v_3 = sgn(e_{v_3 \rightarrow v_1}) \times v_1 + sgn(e_{v_3 \rightarrow v_2}) \times v_2 \quad (9)$$

Therefore, the problem of generating efficient common IPSs in the group can be recast as a problem of partitioning the inputs in the same group into pairs, which are efficient weight-two IPS with the high sharing probability. Suppose there are  $g$  inputs in the group, which are  $a_1, a_2, \dots, a_g$ . Weight-two IPS is defined to be either positive and negative, as  $a_i + a_j$  and  $a_i - a_j$  respectively, where  $i, j \in [1, g]$ . To efficiently list and search for the weight-two IPS, we introduce an IPS matrix.

**Definition 1: IPS matrix** is a triangular matrix with  $g-1$  rows and  $g-1$  columns. The entry  $IPS(m, n-1)$  in the  $m$ th row and the  $(n-1)$ th column represents the number of occurrences of weight-two IPS formed by  $a_m$  and  $a_n$ , where  $m \in [1, g-1]$ ,  $n \in [2, g]$ .

Because we approximately treat the cost of a subtractor to be the same as an adder,  $a_j - a_i$  and  $a_i - a_j$  is regarded as the same negative Weight-two IPS. Therefore, they don't

TABLE I  
THE PROBABILITY OF WEIGHT-TWO IPS

| Weight-two IPS | $p(a_i, a_j)$ | Weight-two IPS | $p(a_i, a_j)$ |
|----------------|---------------|----------------|---------------|
| $(a_1, a_2)$   | 3/17          | $(a_2, a_4)$   | 3/17          |
| $(a_1, a_3)$   | 2/15          | $(a_2, a_5)$   | 2/15          |
| $(a_1, a_4)$   | 3/17          | $(a_3, a_4)$   | 2/15          |
| $(a_1, a_5)$   | 2/15          | $(a_3, a_5)$   | 1/13          |
| $(a_2, a_3)$   | 2/15          | $(a_4, a_5)$   | 2/15          |

need to be distinguished. Knowing that  $a_i + a_j$  and  $a_i - a_j$  can be calculated by using one unified adder/subtractor (UAS) operator [22] which is with less cost than the total of one adder and one subtractor, we let the entries in *IPS matrix* store the sum of positive weight-two IPS  $a_i + a_j$  and negative weight-two IPS  $a_i - a_j$ . For example, we have five inputs in the group, and assume three high-weight IPSs exist which are  $IPS_1 = a_1 + a_2 + a_3 + a_4 + a_5$ ,  $IPS_2 = a_1 - a_2 + a_4 + a_5$  and  $IPS_3 = a_1 - a_2 - a_3 + a_4$ . The initial *IPS matrix* is presented as:

$$IPS\_matrix = \begin{bmatrix} 3 & 2 & 3 & 2 \\ & 2 & 3 & 2 \\ & & 2 & 1 \\ & & & 2 \end{bmatrix}.$$

For instance, the “2” appearing in the 4th row and the 4th column in *IPS matrix* represents that  $(a_4 \pm a_5)$  appears twice in all high-weight IPSs. With the number of occurrence of weight-two IPS stored, the probability  $p(a_i, a_j)$  that  $a_i$  and  $a_j$  are bound to form an IPS is given by:

$$p(a_i, a_j) = \frac{IPS\_matrix(i, j-1)}{\sum_{\forall m \in \{i, j\} \text{ or } n \in \{i, j\}} IPS\_matrix(m, n-1)} \quad (10)$$

where  $\sum_{\forall m \in \{i, j\} \text{ or } n \in \{i, j\}} IPS\_matrix(m, n-1)$  represents the total number of occurrences of weight-two IPSs which involve  $a_i$  and/or  $a_j$ .

We can use (10) to calculate the probability of each combination. The weight-two IPS with the maximum probability  $p(a_i, a_j)$  is first selected to pair up  $a_i$  and  $a_j$ . The number of occurrences of the fixed pairs are reset to 0 in *IPS matrix*. Fixing  $a_i$  and  $a_j$  together cause them to be impossible to pair with other input, so *IPS matrix* is updated to ensure that the probability is calculated based on the latest statistics. After using (10) to compute the probabilities again, new weight-two IPS with the highest probability is selected. The iteration continues until all the entries in *IPS matrix* become zero.

Similarly as the same example above, all  $p(a_i, a_j)$  of weight-two IPS are calculated and shown in TABLE I. There are multiple weight-two IPSs with the maximum probability  $p(a_i, a_j) = 3/17$ . Because pairing  $a_1$  and  $a_4$  requires only one adder, while other pairs require a UAS,  $a_1$  and  $a_4$  are paired into a weight-two IPS first. *IPS matrix* is subsequently updated to reflect the change, as

$$IPS\_matrix = \begin{bmatrix} 0 & 0 & 0 & 0 \\ & 2 & 0 & 2 \\ & & 0 & 1 \\ & & & 0 \end{bmatrix}.$$

The probability of the remaining pairs can be computed as  $p(a_2, a_3) = 2/5$ ,  $p(a_2, a_5) = 2/5$ ,  $p(a_3, a_5) = 1/5$ . Since  $(a_2, a_3)$  has the maximum probability and needs one adder, they will be paired first. With only one input left, no further matching is required. Therefore, by using (10),  $a_1$  and  $a_4$  are paired to become one weight-two IPS, while  $a_2$  and  $a_3$  become another.

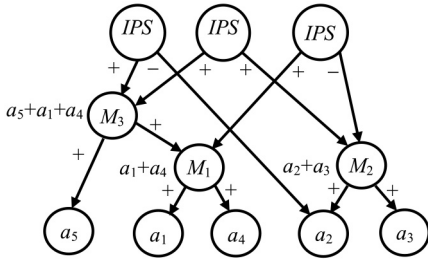


Fig. 4. The complete binary tree of the above example.

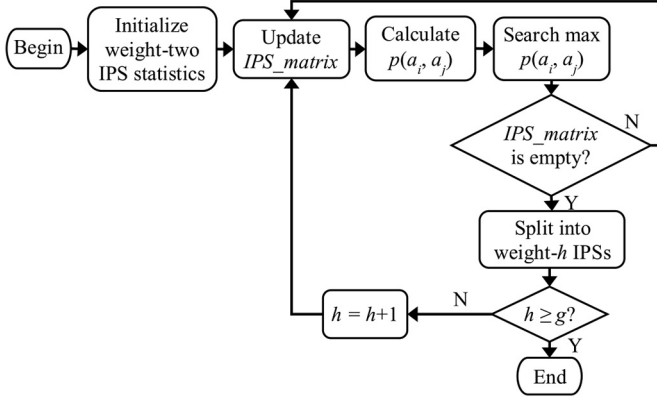


Fig. 5. Flowchart of the proposed IPS sharing.

After all inputs in the same group are paired, all the unique paired weight-two IPS can be treated as the new basis which is similar as the individual input in the weight-two IPS. After this, we use the weight-two IPSs to construct the higher weight IPSs which are sharable. A similar *IPS\_matrix* can be constructed to store the number of occurrences of the high weight IPSs. The search and selection process are the same as the algorithm above used to search and select weight-two IPSs. This high weight IPSs search and sharing continue until weight- $g$  IPS which is the full set of the inputs in this group is constructed. Using the above example, a complete binary tree shown in Fig. 4 is constructed.  $IPS_1$ ,  $IPS_2$  and  $IPS_3$  representing three high weight IPSs are the root vertices. The edges are the signs.  $M_1$ ,  $M_2$  and  $M_3$  are the intermediates.  $(a_1 + a_4)$  and  $(a_2 + a_3)$  are the paired weight-two IPSs.  $(a_1 + a_4 + a_5)$  is the fixed weight-three IPS which can be shared by  $IPS_1$  and  $IPS_2$ .

Fig. 5 outlines the flow for the proposed IPS sharing.  $h$  is the IPS weight which is initialized as 2.  $g$  is the number of inputs in the group. Weight-two IPSs are paired according to their probabilities until *IPS\_matrix* becomes empty. If  $h$  is less than  $g$ , it continues to search for higher weight IPSs.

With the proposed algorithm, the 16-point approximated FFT architecture is depicted in Fig. 6. ①, ② and ③ represent multiplications with  $\cos(\pi/8)$ ,  $\sin(\pi/4)$  and  $\sin(\pi/8)$  respectively. Real-imaginary-combination (RIC) block is to list the outputs. Our design requires less hardware compared with the conventional FFT architecture shown in Fig. 1.

#### IV. EXPERIMENTAL RESULTS AND DISCUSSIONS

FFTs with different orders are designed using the proposed algorithm and compared with the relevant architecture proposed in [3], [8], [18] and [23]. For fair comparison, we compare the hardware cost which can achieve the same error requirement. All adders in both our design and competing

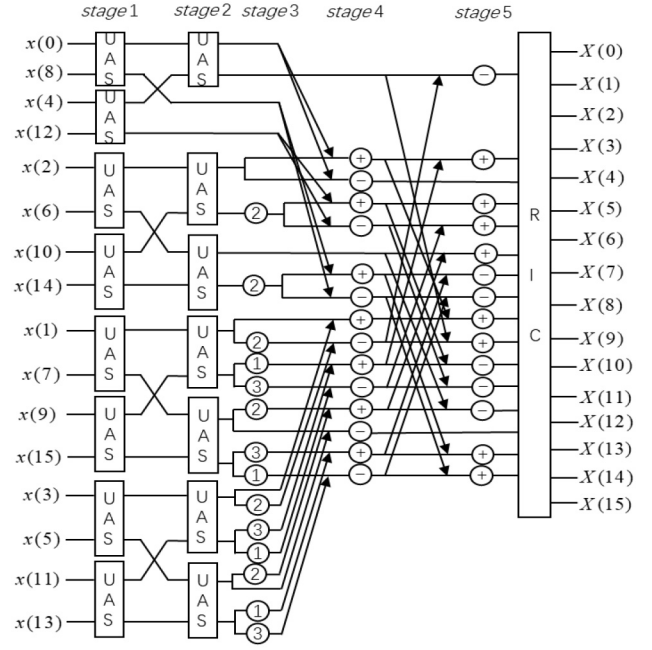


Fig. 6. The proposed 16-point FFT architecture.

TABLE II  
COMPARISON OF THE FFT DESIGNS ON FPGA

| Design   | 16-point FFT |      |            |            | 32-point FFT |      |            |            |
|----------|--------------|------|------------|------------|--------------|------|------------|------------|
|          | #LUT         | #FF  | Power (mW) | Delay (ns) | #LUT         | #FF  | Power (mW) | Delay (ns) |
| Proposed | 1456         | 964  | 118        | 11.126     | 5211         | 2442 | 147        | 17.393     |
| [3]      | 2633         | 1700 | 126        | 17.607     | 12929        | 5917 | 308        | 24.671     |
| [18]     | 1533         | 1349 | 120        | 14.741     | 5552         | 2954 | 152        | 18.82      |
| [8]      | 3095         | 1759 | 156        | 26.608     | 13268        | 5353 | 588        | 30.832     |
| [23]     | 2263         | 2000 | 158        | 26.409     | 6395         | 6643 | 269        | 29.293     |

methods are assumed to be implemented using ripple carry adders. All designs are described in Verilog and synthesized on Xilinx Spartan-7, xc7s75fgga484-1 FPGA device. Xilinx Vivado 2019.2 is used to synthesize the designs. The clock frequency was set to 25 MHz. The number of LUTs, the number of Flip Flops, power and delays are shown in Table II.

Compared with [3], [8], [18] and [23], at least 28.5% and 17.3% savings in terms of #FF are achieved by our 16- and 32-point designs respectively. Our design also reduced #LUTs over other competing methods by at least 5.0% and 6.1% respectively. The reason of these savings is that the proposed IPS sharing scheme can generate more reusable IPSs. For delay, the proposed 16-point has the shortest delay. For 32-point, our delay is also shorter than the most competitive method [18] by 7.6%, because the proposed IPS sharing reduces the number of adders in the critical path. For power, the cost of the proposed 16-point and 32-point designs are lower than [23] by at least 25.32% and 45.35% respectively. Compared with the most competitive designs by [18], our solutions are still with slightly lower power cost.

We compare the performance on ASIC with 45nm cell library and synthesize using DesignCompiler. Synopsys PowerCompiler version: J-2014.09-SP3 is used to analysis the power. Clock rate of 25 MHz and a supply voltage of 1.0V are used. The synthesized areas, power cost and delays are shown in Table III. The improvements by the proposed designs in area, delay and power over [3], [8] and [23] are significant.

TABLE III  
COMPARISON OF THE FFT DESIGNS ON ASIC

| Design   | 16-point FFT             |            |            | 32-point FFT             |            |            |
|----------|--------------------------|------------|------------|--------------------------|------------|------------|
|          | Area ( $\mu\text{m}^2$ ) | Power (mW) | Delay (ns) | Area ( $\mu\text{m}^2$ ) | Power (mW) | Delay (ns) |
| Proposed | 13510                    | 0.9749     | 2.98       | 45574                    | 5.478      | 3.54       |
| [3]      | 23781                    | 1.7366     | 3.36       | 96548                    | 11.585     | 4.89       |
| [18]     | 16701                    | 1.2043     | 3.21       | 46945                    | 5.8845     | 3.74       |
| [8]      | 26188                    | 1.923      | 5.55       | 96125                    | 11.369     | 7.21       |
| [23]     | 22015                    | 1.6064     | 5.14       | 65357                    | 12.775     | 6.52       |

TABLE IV  
COMPARISON BETWEEN THE PROPOSED 64-POINT FFT AND [18]

| 64-point FFT | FPGA  |      |            |            | ASIC                     |            |            |
|--------------|-------|------|------------|------------|--------------------------|------------|------------|
|              | #LUT  | #FF  | Power (mW) | Delay (ns) | Area ( $\mu\text{m}^2$ ) | Power (mW) | Delay (ns) |
| proposed     | 18515 | 3076 | 211        | 21.204     | 116423                   | 8.726      | 3.80       |
| [18]         | 19247 | 4280 | 240        | 22.236     | 128484                   | 9.527      | 4.04       |

TABLE V  
COMPARISON BETWEEN INPUT GROUPING AND IPS SHARING

| 16-point FFT | FPGA |      |            |            | ASIC                     |            |            |
|--------------|------|------|------------|------------|--------------------------|------------|------------|
|              | #LUT | #FF  | Power (mW) | Delay (ns) | Area ( $\mu\text{m}^2$ ) | Power (mW) | Delay (ns) |
| DIT-FFT      | 2271 | 1833 | 142        | 19.282     | 32443                    | 2.3632     | 4.63       |
| IG           | 1623 | 1044 | 120        | 11.504     | 15293                    | 1.1059     | 3.16       |
| IG_IPSS      | 1456 | 964  | 118        | 11.126     | 13510                    | 0.9748     | 2.98       |

Compared with efficient designs by [18], the area cost by the proposed 16-point design is reduced by 19.1%. Besides shorter delays, our 16-point and 32-point designs reduce the power over [18] by 12.2% and 6.9% respectively.

To further verify our improvement over the competitive designs by [18], 64-point FFTs are designed and the results are shown in Table IV. For both FPGA and ASIC, the proposed designs consistently perform better than that presented by [18]. Experiments on higher order FFTs show the scalability of the proposed method with the obvious improvements over [18].

To investigate the effectiveness of each individual technique, we use 16-point FFT as example and develop three 16-point FFTs, which are original DIT-FFT as shown in Fig. 1, FFT using input grouping (IG) only, and FFT using both input grouping and IPS sharing (IG\_IPSS). The results are shown in Table V. For FPGA, IG successfully reduces 28.53% #LUT and 43.04% #FF over original DIT-FFT. The delay and power reduction are 40.34% and 15.49%. This shows the effectiveness of input grouping when it works alone. After IPS sharing is introduced, the #LUT and #FF costs of IG\_IPSS are further reduced by 10.29% and 7.66% respectively over IG. It reduces the delay and power too. For ASIC, our IG reduces 52.8% area over original DIT-FFT. The delay and power of IG are 53.2% and 31.75% lower than DIT-FFT. This proves the effectiveness of the proposed input grouping when working alone. After IPS sharing is introduced, the ASIC area of IG\_IPSS is further reduced by 11.87% over IG.

From the results, we conclude that both input group and IPS sharing are effective. The contribution by input grouping is more significant, because it reduces the multiplication which is the most resource-expensive arithmetic.

## V. CONCLUSION

This brief presents a novel low power and low complexity FFT design. The proposed input grouping provides an efficient

solution to reduce the number of multiplications. Meanwhile, the input partial sum sharing algorithm can further reduce the adder cost. Our designs with different FFT orders on FPGA and ASIC achieve significant improvements compared with the recent competing methods in the literature.

## REFERENCES

- [1] W. Hong *et al.*, "Multibeam antenna technologies for 5G wireless communications," *IEEE Trans. Antennas Propag.*, vol. 65, no. 12, pp. 6231–6249, Dec. 2017.
- [2] P. K. Daram and C.-I. H. Chen, "Smart FFT measurement for reconfigurable sensor using a wideband digital receiver," in *Proc. IEEE Nat. Aerosp. Electron. Conf.*, Aug. 2021, pp. 200–203.
- [3] J. Chen, J. Hu, S. Lee, and G. E. Sobelman, "Hardware efficient mixed radix-25/16/9 FFT for LTE systems," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 23, no. 2, pp. 221–229, Feb. 2015.
- [4] N. Lei and X. Gu, "FFT-OT: A fast algorithm for optimal transportation," in *Proc. IEEE/CVF ICCV*, Oct. 2021, pp. 6260–6269.
- [5] J. W. Cooley and J. W. Tukey, "An algorithm for the machine calculation of complex Fourier series," *Math. Comput.*, vol. 19, pp. 297–301, Apr. 1965.
- [6] J. Wang, S. Li, and X. Li, "Scheduling of data access for the radix-2k FFT processor using single-port memory," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 28, no. 7, pp. 1676–1689, Jul. 2020.
- [7] J. Hazarika, M. T. Khan, and S. R. Ahamed, "Low-complexity continuous-flow memory-based FFT architectures for real-valued signals," in *Proc. 32nd Int. Conf. VLSI Des. 18th Int. Conf. Embedded Syst. (VLSID)*, Jan. 2019, pp. 46–51.
- [8] S. Liu and D. Liu, "A high-flexible low-latency memory-based FFT processor for 4G, WLAN, and future 5G," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 27, no. 3, pp. 511–523, Mar. 2019.
- [9] T. Kawabata and H. Tsutsui, "An evaluation of high-throughput scalable radix-4 FFT processor architecture using fixed-point arithmetic," in *Proc. Asia-Pacific Signal Inf. Process. Assoc. Annu. Summit Conf. (APSIPA ASC)*, Dec. 2020, pp. 114–117.
- [10] N. Nadar, M. Mehta, K. Bhat, J. Mendes, and R. Chaudhari, "Hardware implementation of pipelined FFT using polyphase decomposition," in *Proc. Int. Conf. Recent Trends Electron. Inf. Commun. Technol.*, Aug. 2021, pp. 339–343.
- [11] M. Parfeniuk, "Lifting-based alternatives to the FFT for computing the 4-, 8-, and 16-point discrete Fourier transforms," in *Proc. 29th Eur. Signal Process. Conf.*, Aug. 2021, pp. 2184–2188.
- [12] *IEEE Standard for Information Technology—Telecommunications and Information Exchange Between Systems—Local and Metropolitan Area Networks—Specific Requirements—Part 11: Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) Specifications Amendment 3: Enhancements for Very High Throughput in the 60 GHz Band*, IEEE Standard 802.11ad-2012, Dec. 2012.
- [13] B. Liu *et al.*, "Precision adaptive MFCC based on R2SDF-FFT and approximate computing for low-power speech keywords recognition," *IEEE Circuits Syst. Mag.*, vol. 21, no. 4, pp. 24–39, 4th Quart., 2021.
- [14] H.-Y. Chin, P.-Y. Tsai, and S.-Y. Lee, "Implementation of a two-dimensional FFT/IFFT processor for real-time high-resolution synthetic aperture radar imaging," in *Proc. IEEE Workshop Signal Process. Syst. (SiPS)*, Oct. 2021, pp. 211–216.
- [15] S.-C. Hsu, S.-J. Huang, S.-G. Chen, S.-C. Lin, and M. Garrido, "A 128-point multi-path SC FFT architecture," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, Oct. 2020, pp. 1–5.
- [16] M. Garrido and P. Paz, "Optimum MDC FFT hardware architectures in terms of delays and multiplexers," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 68, no. 3, pp. 1003–1007, Mar. 2021.
- [17] X. Han, J. Chen, and S. Rahardja, "A new twiddle factor merging method for low complexity and high speed FFT architecture," in *Proc. IEEE Int. Circuits Syst. Symp.*, Sep. 2019, pp. 1–5.
- [18] X. Han, J. Chen, B. Qin, and S. Rahardja, "A novel area-power efficient design for approximated small-point FFT architecture," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 39, no. 12, pp. 4816–4827, Dec. 2020.
- [19] T.-W. Siu, C.-W. Sham, and F. C. M. Lau, "Operating frequency improvement on FPGA implementation of a pipeline large-FFT processor," in *Proc. 19th ICACT*, Feb. 2017, pp. 5–9.
- [20] T.-T. Hoang, D.-H. Le, and C.-K. Pham, "VLSI design of floating-point twiddle factor using adaptive CORDIC on various iteration limitations," in *Proc. IEEE 12th Int. Symp. Embedded Multicore/Many-Core Syst. Chip*, Sep. 2018, pp. 225–232.
- [21] W. M. Gentleman and G. Sande, "Fast Fourier transforms: For fun and profit," in *Proc. AFIPS*, San Francisco, CA, USA, Nov. 1966, pp. 563–578.
- [22] J. Ding, J. Chen, and C.-H. Chang, "A new paradigm of common sub-expression elimination by unification of addition and subtraction," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 35, no. 10, pp. 1605–1617, Oct. 2016.
- [23] M. Garrido and P. Malagón, "The constant multiplier FFT," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 68, no. 1, pp. 322–335, Jan. 2021.